

Plano de execucao

Otimizacao de performance

Documento gerado em 14 de maio de 2026

RESUMO EXECUTIVO

O AxeCloud tem 4 gargalos identificados que somados respondem pela percepcao de lentidao ao abrir paginas: bundle inicial grande, falta de indices em tabelas quentes, ausencia de cache HTTP em endpoints estaveis e ausencia de cache/dedup no cliente. Este documento descreve um plano em 6 fases independentes, com estimativa de esforco e impacto, para resolver cada um deles.

Total de mexida: ~2.5-3 h. Ganho composto esperado: a aplicacao 3-5x mais "snappy" no fluxo real de uso.

INDICE

- Fase 1.** Indices SQL
- Fase 2.** Code splitting das views
- Fase 3.** Cache HTTP em endpoints estaveis
- Fase 4.** select() enxuto + paralelismo
- Fase 5.** PWA cache strategy ajustado
- Fase 6.** SWR global nas views

FASE 01

Indices SQL

Ganho: ganho 50-500 ms por query

Arquivo: [supabase/migrations/20260514200000_performance_indexes.sql](#)

Migration nova com indexes para tabelas mais lidas. Atualmente faltam indexes criticos em perfil_lider, filhos_de_santo, financeiro, mural_aviso, calendario_ave e subscriptions - todas usadas em quase todas as paginas.

DETALHAMENTO

Tabela	Index	Por que
perfil_lider	(tenant_id)	Lookup em todas queries de tenant
perfil_lider	(id, tenant_id) composto	OR de resolveTenantFromSupabase
filhos_de_santo	(tenant_id)	Listagem em Children
filhos_de_santo	(lider_id)	Vinculo filho->zelador
filhos_de_santo	(user_id)	loadAllTenantData (filho)
filhos_de_santo	(email)	Fallback por email em login
financeiro	(tenant_id, data DESC)	Listagens ordenadas
financeiro	(tenant_id, status)	Filtro pendente/pago
mural_aviso	(tenant_id, created_at DESC)	NoticeBoard e Painel de Notificacoes
calendario_ave	(tenant_id, data DESC)	Calendar e Dashboard
calendario_ave	(tipo)	Filtro Obrigacao
subscriptions	(user_id)	Settings e Subscription

ACAO MANUAL NECESSARIA

Rodar a migration manualmente no Supabase (dashboard > SQL editor) ou via supabase CLI.

RISCO

Zero risco. Indexes usam CREATE INDEX IF NOT EXISTS e CONCURRENTLY onde aplicavel - nao bloqueiam leituras nem escritas.

FASE 02

Code splitting das views

Ganho: ganho 1-3 s no primeiro carregamento

Arquivo: [src/App.tsx \(~30 linhas alteradas\)](#)

Trocar imports estaticos linhas 4-19 por React.lazy + Suspense. O bundle inicial cai de ~900 KB para ~250 KB. Cada view vira chunk separado de 30-80 KB carregado on-demand.

DETALHES

- Logged-in usuario ve Dashboard em ~1.2 s vs ~3 s hoje (rede 4G media)
- Cada view e cacheada no navegador na 2a abertura - troca de aba instantanea
- Login, Sidebar, NotificationPanel e LuxuryLoading nao viram lazy (sao always-on)
- Recharts pode ser lazy dentro de Dashboard/Financial (corta mais 120 KB)

RISCO

Baixo. Posso adicionar prefetch no hover do menu da sidebar para evitar qualquer flicker ao trocar de aba.

FASE 03

Cache HTTP em endpoints estaveis

Ganho: ganho 200-500 ms por navegacao

Arquivo: `api/index.ts` e `api/tenant-info.ts`

Adicionar Cache-Control em endpoints cujo conteudo e estavel por segundos/minutos. Hoje todas as chamadas pagam ~200-600 ms de cold start + ~50-150 ms de createClient Supabase + RLS.

DETALHES

- NAO cachear: `/api/transactions`, `/api/auth/*`, `/api/admin/*`, mutations (POST/PUT/DELETE)
- s-maxage = cache no CDN da Vercel (edge - ~10 ms vs ~500 ms)
- swr = stale-while-revalidate: serve cache na hora, revalida em background

DETALHAMENTO

Endpoint	Cache-Control	Estabilidade
<code>/api/tenant-info</code>	private, max-age=30, swr=120	Plano/tenant muda raramente
<code>/api/plans-catalog</code>	public, s-maxage=300, swr=3600	Catalogo global - cacheado no edge Vercel
<code>/api/v1/financial/pix-config</code>	private, max-age=60, swr=300	Config manual
<code>/api/children</code>	private, max-age=10, swr=60	Lista de filhos relativamente estavel

RISCO

Baixo. stale-while-revalidate garante que dado eventualmente revalida. Tempos curtos (10-300s).

FASE 04

select() enxuto + paralelismo

Ganho: ganho 100-300 ms por tela com muitos dados

Arquivo: `views/Children.tsx`, `Calendar.tsx`, `Library.tsx`, `NoticeBoard.tsx` + `App.tsx::`

Refatorar queries que usam `select("*")` em listagens. Trafegar so o necessario reduz payload em 60-80% em telas com muitos registros, alem de baixar JSON parse e tempo de transmissao em rede movel.

DETALHAMENTO

Arquivo	Hoje	Depois
Children.tsx	<code>select("*")</code>	<code>select("id, nome, foto_url, cargo, ativo, created_at")</code>
Calendar.tsx	<code>select("*")</code>	so campos da listagem
Library.tsx	<code>select("*")</code>	so metadados (sem blob)
NoticeBoard.tsx	<code>select("*")</code>	sem comentarios (lazy ao expandir)
loadAllTenantData	queries sequenciais	Promise.all paralelo

RISCO

Medio. Preciso garantir que nenhum componente downstream usa coluna removida do select. Faco grep cuidadoso antes de cada mudanca.

FASE 05

PWA cache strategy ajustado

Ganho: ganho 50-150 ms em revisitas (assets servidos do cache local em vez de revalidar)

Arquivo: vite.config.ts

Hoje TUDO usa NetworkFirst. Isso revalida assets imutaveis em cada navegacao desnecessariamente. Diferenciar por tipo de recurso e mais eficiente.

DETALHAMENTO

Tipo de asset	Estrategia ideal
JS/CSS com hash em /assets/	CacheFirst (1 ano)
Imagens estaticas (icones PWA)	CacheFirst (30 dias)
Navegacao HTML	NetworkFirst (continua igual)
Chamadas /api/*	NetworkOnly (deixa Cache-Control HTTP decidir)
Imagens de usuario (R2/Supabase)	StaleWhileRevalidate (1 dia)

RISCO

Baixo. Assets com hash sao imutaveis por construcao - CacheFirst e seguro.

FASE 06

SWR global nas views

Ganho: ganho de PERCEPCAO - aplicacao parece instantanea no fluxo do dia-a-dia

Arquivo: views/Children, Calendar, NoticeBoard, Lib

Trocar useEffect + fetch + useState por useSWR. Hoje so o Financial.tsx usa SWR - o resto refetch tudo a cada troca de aba.

DETALHES

- Dedup automatico: 2 componentes pedindo o mesmo dado -> 1 request
- Stale-while-revalidate: dado cacheado aparece na hora, revalida em background
- Revalidate on focus: dado fresco sem o usuario apertar reload
- Cache compartilhado entre Dashboard e Financial (transacoes sao as mesmas)
- Remove boilerplate de loading/error em cada view

RISCO

Medio. Mexe em mais arquivos. Faco uma view por vez com teste manual entre cada.

Cronograma sugerido

As fases são independentes e podem ser executadas em paralelo ou em ordens diferentes. Recomendo seguir a ordem do plano - cada fase entrega valor isolado e a primeira já reduz drasticamente o tempo de abertura de páginas.

Fase	Esforço	O que você faz	Impacto
1. Índices SQL	10 min	rodar 1 migration no Supabase	5/5
2. Code splitting	20 min	nada (deploy automático)	5/5
3. Cache HTTP	15 min	nada	4/5
4. Select enxuto	30 min	nada	3/5
5. PWA strategy	10 min	nada	2/5
6. SWR global	1-2 h	testar visualmente cada view	4/5

Recomendações de execução

- Fast track (Fases 1-3): ~45 min de trabalho, entrega ~70% do ganho. So mexe em infra (migration SQL + bundling + headers HTTP).
- Completo (Fases 1-5 hoje, Fase 6 depois): dá espaço para testar cada view do SWR com calma.
- Customizado: você escolhe quais fases - todas são independentes e já existem revisões incrementais sem efeito colateral.

Como medir o ganho

- Lighthouse / PageSpeed Insights: rodar antes/depois de cada fase. Métricas a observar: LCP, TTI, TBT.
- Network tab do DevTools: tempo de "DOMContentLoaded" cai diretamente após Fase 2.
- Supabase dashboard > Database > Query Performance: tempo médio de queries cai após Fase 1.
- Vercel Speed Insights (já instalado): métricas reais de usuário chegam em 24-48h após deploy.

